The most basic method is find. The find method takes a block and returns either the first object for which the block returns a truthy value or nil if there is no such object. If you'd rather have the index of the object, you can call find_index with the same behavior. It's worth pointing out that find and find_index both take optional arguments, but they take different optional arguments. For find, the argument is a proc that is called and its value returned in lieu of returning nil if no matching object is found. For find_index the argument replaces the block and the method returns the index of the first object in the list that is == to the argument.

Often you may want to return all objects in a list that match a given block, and Enumerator provides that behavior under three different aliases: find_all, filter, and select. These are all names for the same behavior: take a block and return every element of the list for which that block returns a truthy value. The inverse is reject, which returns every element of the block for which the block returns a falsey value.

The similar method grep takes a pattern argument, not a block, and returns every element of the array that is === the argument. It's inverse, grep_v returns every element of the array that is *not* === the argument. If there's a block, then each matching element that would be returned by the method is passed to the block and the return value of the block is used instead. So:

```
list = [1, 1, 2, 3, 5, 8, 13, 21]
list.find { |i| i.even? }                  # => 2
list.find(proc { "nope" }) { |i| i.zero? } # => "nope"

list.find_index { |i| i.even? }            # => 2
list.find_index(5)                         # => 4

list.select { |i| i.even? }                # => [2, 8]
list.reject { |i| i.even? }                # => [1, 1, 3, 5, 13, 21]

list.grep(4..10)                           # => [5, 8]
list.grep_v(4..10)                         # => [1, 1, 2, 3, 13, 21]

list.grep(4..10) { |x| 10 ** x }           # => [100000, 100000000]
```

Two common filter patterns have their own methods: compact and uniq. compact removes all nil elements from an array, and is particularly useful to clean up arrays before passing them forward to map or sum or something else that might break if passed a nil.

The uniq method removes duplicate elements (based on eql? or ==). Normally uniq is based on the actual elements of a collection, but with a block, it will compare values based on the return value of the block for each element:

```
[1, 2, nil, 3, nil, 4, 5].compact # => [1, 2, 3, 4, 5]
[1, 2, 3, 2, 3, 1].uniq           # => [1, 2, 3]
```

You might want to split an enumerable into sub-lists based on some criteria, similar to the way split works for strings. For Enumerable, you can do this with slice_after, slice_before, and slice_when. All three of these methods return Enumerator instance rather than arrays.

The slice_after method takes a pattern or a block. It tests each element of the collection in turn. If the element is either === to the argument or causes the block to return a truthy value, then that element is the end of a slice and a new slice begins with the next element.

The slice_before method performs the same test but if an element matches, then it ends the slice before that element and the element is the first element of the next slice.